

AA2-EV02 - Despliegue de contenedores Docker

- [AA2-EV02 — Taller de aplicación: ejercicio de despliegue de contenedores](#)
 - [1. Introducción](#)
 - [2. Objetivos](#)
 - [3. Requerimientos del entorno](#)
 - [4. Arquitectura tecnológica](#)
 - [5. FASE 00 — Diagnóstico del equipo](#)
 - [6. FASE 01 — Preparación del entorno y de la evidencia](#)
 - [7. FASE 02 — Verificación de Docker](#)
 - [8. FASE 03 — Obtención y análisis de la aplicación](#)
 - [9. FASE 04 — Dockerfile](#)
 - [10. FASE 05 — Construcción de la imagen](#)
 - [11. FASE 06 — Verificación de la imagen](#)
 - [12. FASE 07 — Creación y ejecución del contenedor](#)
 - [13. FASE 08 — Verificación de la aplicación](#)
 - [14. FASE 09 — Modificación de la aplicación](#)
 - [15. FASE 10 — Reconstrucción de la imagen](#)
 - [16. FASE 11 — Reemplazo del contenedor anterior](#)
 - [17. FASE 12 — Nuevo contenedor y validación de la modificación](#)
 - [18. FASE 13 — Docker Hub](#)
 - [19. FASE 14 — Pruebas finales \(verificación automatizada\)](#)
 - [20. Matriz de cumplimiento](#)
 - [21. Resultados obtenidos](#)
 - [22. Conclusiones](#)
 - [23. Anexos](#)

AA2-EV02 — Taller de aplicación: ejercicio de despliegue de contenedores

Servicio Nacional de Aprendizaje — SENA

Dato	Valor
Programa	Despliegue de aplicaciones y servicios en contenedores Docker
Resultado de aprendizaje	220501086-02 — Crear la infraestructura y plataforma tecnológica según los requerimientos del proyecto de Software
Actividad de aprendizaje	AA2 — Generar un ambiente de desarrollo a partir de la ejecución de imágenes para la creación de contenedores
Evidencia	AA2-EV02 — Taller de aplicación: ejercicio despliegue de contenedores
Aprendiz	Javier Parra
Fecha	7 de octubre de 2026
Entorno	Windows 11 + WSL2 (Ubuntu 24.04.4 LTS), x86_64

1. Introducción

Docker es una plataforma de contenedores que permite empaquetar una aplicación junto con todas sus dependencias en una unidad portátil y reproducible. Los conceptos centrales son:

- **Imagen:** plantilla de solo lectura, inmutable y versionada, que contiene el sistema de archivos y la configuración necesarios para ejecutar una aplicación.
- **Contenedor:** instancia en ejecución de una imagen. Es un proceso aislado con su propio sistema de archivos, red y espacio de procesos.
- **Dockerfile:** archivo de texto con las instrucciones que describen cómo construir una imagen (imagen base, dependencias, código, comando de inicio).
- **Docker Engine:** servicio que construye y ejecuta los contenedores; en este taller se ejecuta a través de **Docker Desktop** sobre **WSL2**.
- **Entorno de desarrollo:** conjunto de herramientas y servicios (código fuente, gestor de dependencias, motor de contenedores, puertos) que permite construir, ejecutar y probar la aplicación de forma aislada y controlada.

Esta práctica despliega una aplicación **Node.js + Express + React** (lista de tareas) dentro de un contenedor, la verifica en el navegador, **modifica su código**, reconstruye la imagen, reemplaza el contenedor y comprueba de extremo a extremo que el cambio llega hasta la aplicación servida.

2. Objetivos

Objetivo general

Desplegar una aplicación en un contenedor Docker a partir de un Dockerfile, verificando su funcionamiento y demostrando el ciclo completo **código → imagen → contenedor → puerto → aplicación**, incluida la modificación y reconstrucción de la aplicación.

Objetivos específicos

1. Preparar el entorno y la arquitectura tecnológica (Docker Desktop + WSL2).
2. Verificar el correcto funcionamiento del motor Docker.
3. Construir una imagen propia a partir de un Dockerfile.
4. Ejecutar el contenedor y exponer la aplicación en el puerto 3000.
5. Comprobar el funcionamiento funcional de la aplicación (CRUD).
6. Modificar el código de la aplicación, reconstruir la imagen y reemplazar el contenedor.
7. Documentar la evidencia y demostrar los cinco indicadores de la lista de chequeo.

3. Requerimientos del entorno

Componente	Detalle real verificado
Sistema operativo	Windows 11 con WSL2
Distribución	Ubuntu 24.04.4 LTS (kernel 6.18.33.2-microsoft-standard-WSL2)
Arquitectura	x86_64
Procesador	12th Gen Intel(R) Core(TM) i7-12650H (16 vCPU)
Memoria RAM	7.6 GiB
Almacenamiento libre	~933 GB
Motor de contenedores	Docker Desktop — Docker Engine 29.7.2
Docker Compose	v5.3.1
Git	2.43.0
Node.js	v24.19.0 (host)
npm	11.17.0

Componente	Detalle real verificado
Editor	Visual Studio Code 1.139.1

4. Arquitectura tecnológica

```
Windows 11 (host)
|
v
WSL2 - Ubuntu 24.04 ---> Docker Desktop
                        |
                        v
                        Docker Engine 29.7.2
                        |
                        v
                        Imagen: getting-started:latest
                        |
                        v
                        Contenedor: getting-started
                        |
                        v
                        Puerto 3000 (host -> contenedor)
                        |
                        v
                        Navegador: http://localhost:3000
```

La aplicación se compone de: `src/index.js` (servidor Express, escucha en el puerto 3000), `src/routes/` (endpoints REST `/items`), `src/persistence/` (SQLite) y `src/static/` (interfaz React). El contenedor aísla el runtime Node.js y sus dependencias del resto del sistema.

5. FASE 00 — Diagnóstico del equipo

Objetivo. Determinar el estado real del entorno antes de instalar o modificar cualquier cosa.

Procedimiento. Se ejecutaron comprobaciones no destructivas de sistema operativo, hardware, Docker, Git, Node.js y editor.

Comandos.

```
uname -a; cat /etc/os-release; arch  
lscpu | grep -E "Model name|^CPU(s)"; free -h; df -h /  
docker --version; docker info; docker compose version  
git --version; node --version; npm --version; code --version
```

Resultado. Docker Desktop estaba instalado pero con el motor detenido inicialmente; tras activarlo, `docker --version` devolvió `Docker version 29.7.2` y `docker info` mostró un servidor operativo con `Storage Driver: overlayfs`, 16 CPU y 7.59 GiB de memoria.

Evidencia. `evidencias/00-diagnostico/diagnostico-equipo.txt`.

Figura 1. Diagnóstico del equipo: sistema operativo WSL2, hardware y herramientas disponibles.

Análisis técnico. El motor Docker es el componente que realmente construye y ejecuta contenedores; el cliente `docker` sólo lo controla. Confirmar que el servidor responde (no solo que el cliente existe) es requisito indispensable antes de continuar.

Criterio SENA. Aporta a CHECK-02 (conceptos de contenedores) y a la preparación de la arquitectura tecnológica.

6. FASE 01 — Preparación del entorno y de la evidencia

Objetivo. Estructurar el proyecto y el sistema de trazabilidad de evidencias.

Procedimiento. Se creó la estructura de carpetas `app/` , `docker/` , `scripts/` , `evidencias/00-diagnostico ... 14-final` , `logs/` , `metadata/` y `documento/` . Se inicializaron `metadata/project-state.json` , `metadata/commands.log` , `metadata/evidence-index.json` y `metadata/checklist.json` .

Resultado. Estructura de evidencia organizada cronológicamente y estado del proyecto en formato JSON.

7. FASE 02 — Verificación de Docker

Objetivo. Comprobar que el motor Docker funciona correctamente.

Comandos.

```
docker --version
docker compose version
docker info
```

Resultado.

```
Docker version 29.7.2, build a7dcaa6
Docker Compose version v5.3.1
Server:
  Server Version: 29.7.2
  Storage Driver: overlayfs
  Operating System: Docker Desktop
  Architecture: x86_64
```

CPUs: 16

Total Memory: 7.591GiB

Evidencia. evidencias/02-docker/docker-verificacion.txt ,
evidencias/02-docker/docker-info.txt , evidencias/02-docker/
docker-version-info.png , evidencias/02-docker/docker-info-
server.png .

```
javier@JavierDev:~$ docker --version
Docker version 29.7.2, build a7dcaab
javier@JavierDev:~$ docker info
Client:
Version:      29.7.2
Context:      default
Debug Mode:   false
Plugins:
agent: Docker AI Agent Runner (Docker Inc.)
  Version: v1.119.0
  Path: /usr/local/lib/docker/cli-plugins/docker-agent
ai: Docker AI Agent - Ask Gordon (Docker Inc.)
  Version: v1.30.0
  Path: /usr/local/lib/docker/cli-plugins/docker-ai
buildx: Docker Buildx (Docker Inc.)
  Version: v0.36.0-desktop.1
  Path: /usr/local/lib/docker/cli-plugins/docker-buildx
compose: Docker Compose (Docker Inc.)
  Version: v5.3.1
  Path: /usr/local/lib/docker/cli-plugins/docker-compose
debug: Get a shell into any image or container (Docker Inc.)
  Version: 0.0.47
  Path: /usr/local/lib/docker/cli-plugins/docker-debug
desktop: Docker Desktop commands (Docker Inc.)
  Version: v0.4.3
  Path: /usr/local/lib/docker/cli-plugins/docker-desktop
dhi: CLI For managing Docker Hardened Images (Docker Inc.)
  Version: v0.0.7
  Path: /usr/local/lib/docker/cli-plugins/docker-dhi
extension: Manages Docker extensions (Docker Inc.)
  Version: v0.2.31
  Path: /usr/local/lib/docker/cli-plugins/docker-extension
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version: v1.4.0
  Path: /usr/local/lib/docker/cli-plugins/docker-init
mcp: Docker MCP Plugin (Docker Inc.)
  Version: v0.45.3
  Path: /usr/local/lib/docker/cli-plugins/docker-mcp
model: Docker Model Runner (Docker Inc.)
  Version: v1.2.6
  Path: /usr/local/lib/docker/cli-plugins/docker-model
offload: Docker Offload (Docker Inc.)
  Version: v0.6.9
  Path: /usr/local/lib/docker/cli-plugins/docker-offload
pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
  Version: v0.2.0
  Path: /usr/local/lib/docker/cli-plugins/docker-pass
sandbox: "docker sandbox" is deprecated, use Docker Sandboxes instead (Docker Inc.)
  Version: v0.13.0
  Path: /usr/local/lib/docker/cli-plugins/docker-sandbox
scout: Docker Scout (Docker Inc.)
  Version: v1.24.0
  Path: /usr/local/lib/docker/cli-plugins/docker-scout
Server:
Containers: 4
  Running: 1
  Paused: 0
  Stopped: 3
Images: 15
Server Version: 29.7.2
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
```

Figura 2. Verificación del motor Docker: `docker --version` (29.7.2) y `docker info` (cliente y plugins).

```

Server:
Containers: 4
  Running: 1
  Paused: 0
  Stopped: 3
Images: 15
Server Version: 29.7.2
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Discovered Devices:
  cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: nvidia runc io.containerd.runc.v2
Default Runtime: runc
Init Binary: docker-init
containerd version: e53c7c1516c3b2bff98eb76f1f4117477e6f4e66
runc version: v1.3.6-0-g491b69ba
init version: de40ad0
Security Options:
  seccomp
    Profile: builtin
  cgroupns
Kernel Version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 16
Total Memory: 7.59GiB
Name: docker-desktop
ID: a4ae649d-1f08-4dd1-b1b0-66d8c63df375
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
  com.docker.desktop.address=unix:///var/run/docker-cli.sock
Experimental: false
Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables

javier@JavierDev:~$

```

Figura 3. `docker info` — sección del servidor: `Server Version 29.7.2` , `Storage Driver: overlayfs` , `Operating System: Docker Desktop` , `Architecture: x86_64` , `16 CPU`.

Análisis técnico. `docker info` consulta al daemon; su respuesta confirma que el servidor está activo y describe el almacenamiento, la red y la arquitectura. Docker Compose está disponible como plugin (`docker compose`), aunque este taller no lo requiere porque se usa un único contenedor.

Criterio SENA. CHECK-01 (navegación en Docker) y CHECK-02 (Docker Engine).

8. FASE 03 — Obtención y análisis de la aplicación

Objetivo. Obtener la aplicación de referencia y analizar su estructura sin modificarla.

Procedimiento. Se clonó el repositorio oficial de la aplicación del tutorial de Docker.

Comandos.

```
git clone --depth 1 https://github.com/docker/getting-started-app.git
```

Resultado. Aplicación obtenida en el commit `6b025fc53bc7b9bef435d6b09bcd1da5a871c9cc`.

Elemento	Valor
Nombre (<code>package.json</code>)	<code>101-app</code>
Dependencias	<code>express ^5.2.1 , sqlite3 ^5.1.7 , mysql2 ^3.16.1 , uuid ^13.0.0</code>
Punto de entrada	<code>src/index.js</code>
Puerto	<code>3000 (app.listen(3000))</code>
Interfaz	<code>src/static/index.html + src/static/js/app.js (React)</code>
Persistencia	<code>SQLite (src/persistence/sqlite.js)</code>

Evidencia. `evidencias/03-aplicacion/app-estructura.txt`.

Figura 4. Estructura y análisis de la aplicación de referencia.

Análisis técnico. `src/index.js` monta el servidor Express, sirve los archivos estáticos y expone los endpoints REST `/items`. La interfaz

es una SPA React que consulta esos endpoints. Identificar el puerto (3000) y el punto de entrada es imprescindible para escribir correctamente `EXPOSE` y `CMD` en el Dockerfile.

Criterio SENA. CHECK-04 (aplicación y entorno de desarrollo).

9. FASE 04 — Dockerfile

Objetivo. Definir la receta de construcción de la imagen.

Procedimiento. Se creó `docker/Dockerfile`.

Dockerfile.

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Evidencia. `evidencias/04-dockerfile/dockerfile-evidence.txt`.

Análisis técnico. La imagen base `node:22-alpine` es una versión **LTS** de Node.js sobre Alpine Linux (imagen reducida). El material del SENA referencia `node:12-alpine`, que **está fuera de soporte (EOL)**; se usó la versión LTS vigente por compatibilidad con `express 5` y `sqlite3 5.x`. Copiar primero `package.json` y `package-lock.json` y luego el código permite aprovechar la caché de capas: las dependencias sólo se reinstalan si cambian los manifiestos. `npm ci` garantiza una instalación reproducible a partir del lockfile. `CMD` usa la forma `exec ( ["node", "src/index.js"] )`, que evita un shell intermedio.

Criterio SENA. CHECK-03 (imagen), CHECK-02 (Dockerfile).

Diferencia respecto al material: versión de Node 12 (material) → 22 (práctica). Motivo: EOL del 12 y compatibilidad de dependencias.

10. FASE 05 — Construcción de la imagen

Objetivo. Construir la imagen a partir del Dockerfile.

Comando.

```
docker build -f docker/Dockerfile -t getting-started app/  
getting-started-app
```

Resultado. La construcción terminó con `EXIT=0`. Se descargó la imagen base `node:22-alpine` (Node 22.23.3), se instalaron 244 paquetes con `npm ci` y se generó `getting-started:latest` con ID `70d582dbf294`.

Evidencia. `evidencias/05-build/docker-build.txt`.

Figura 5. Proceso de `docker build` finalizado correctamente.

Análisis técnico. BuildKit ejecuta cada instrucción del Dockerfile como una capa. `-f docker/Dockerfile` permite mantener el Dockerfile separado del código fuente (buena práctica), y `.` / `app/getting-started-app` define el **contexto de construcción** (los archivos disponibles para `COPY`).

Criterio SENA. CHECK-03 (genera con éxito la imagen).

11. FASE 06 — Verificación de la imagen

Objetivo. Confirmar que la imagen existe y registrar sus metadatos.

Comando.

```
docker image ls getting-started
docker image inspect getting-started
```

Resultado.

Repository	Tag	Image ID	Tamaño	Arquitectura
getting-started	latest	70d582dbf294 (v1)	81.5 MB (contenido)	linux/amd64

Evidencia. evidencias/06-imagen/docker-image-ls.txt .

Figura 6. Imagen getting-started creada correctamente.

Análisis técnico. `docker image ls` lista las imágenes almacenadas localmente. El Image ID es el hash del manifiesto de la imagen; identificarlo permite distinguir la versión inicial de la reconstruida tras la modificación.

Criterio SENA. CHECK-01 y CHECK-03.

12. FASE 07 — Creación y ejecución del contenedor

Objetivo. Ejecutar un contenedor a partir de la imagen y exponer la aplicación.

Comando.

```
docker run -dp 3000:3000 --name getting-started getting-started
```

Resultado. Se creó el contenedor `050918f28eba` . `docker ps` mostró el contenedor `Up` con el mapeo `0.0.0.0:3000->3000/tcp` . Los logs indicaron `Listening on port 3000` .

Evidencia. `evidencias/07-contenedor/docker-run-ps.txt` ,
`evidencias/07-contenedor/docker-logs.txt` .

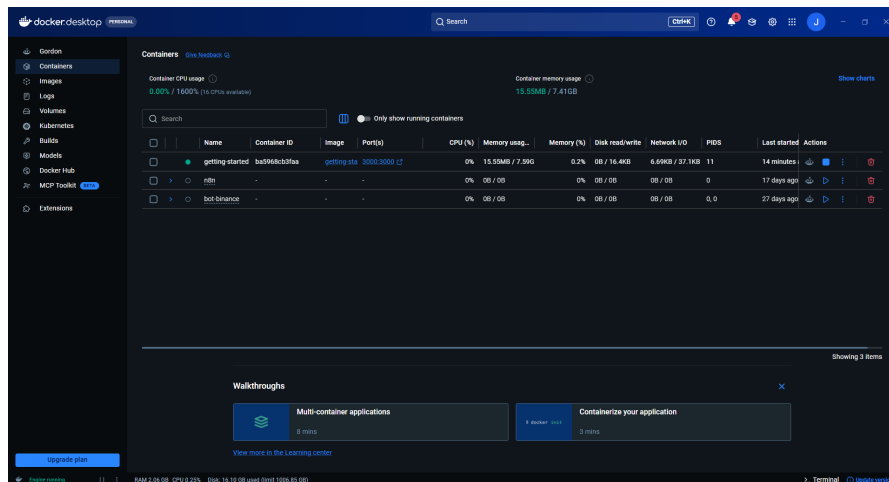


Figura 7. Docker Desktop — el contenedor `getting-started` (`ba5968cb3faa`) en ejecución, con el puerto publicado `3000:3000` .

Análisis técnico. `docker run` crea e inicia un contenedor. `-d` lo ejecuta en segundo plano (detached). `-p 3000:3000` publica el puerto: el primer valor es el puerto del **host** y el segundo el del **contenedor**, de modo que `localhost:3000` en el host llega al puerto 3000 del contenedor. Aislar la aplicación en su propio network namespace y publicar sólo el puerto necesario es parte del diseño seguro de la arquitectura.

Criterio SENA. CHECK-04, CHECK-02 y CHECK-01.

13. FASE 08 — Verificación de la aplicación

Objetivo. Comprobar que la aplicación funciona de verdad (no sólo que el contenedor arranca).

Procedimiento. Se accedió a `http://localhost:3000` y se probó el CRUD completo por HTTP.

Comandos y resultado.

```
GET / -> HTTP 200 | 858 bytes | text/html | <title>Todo App</title>
GET /items -> [] (estado inicial vacío)
POST /items -> {"id":"1ea4...", "name":"Estudiar Docker"}
POST /items -> {"id":"65d7...", "name":"Desplegar contenedor"}
PUT /items/:id -> {"completed":true}
DELETE /items/:id -> OK
```

Evidencia. `evidencias/08-pruebas/app-funcional.txt` .

Figura 8. Aplicación en funcionamiento en `http://localhost:3000` (interfaz React, verificada por HTTP).

Nota de honestidad: la verificación funcional aquí documentada se realizó por HTTP (código de estado y respuestas reales). La captura del navegador de la aplicación desplegada se muestra en la **Figura 12**.

Análisis técnico. Probar la API con `curl` demuestra que el servicio dentro del contenedor atiende peticiones reales y persiste datos (SQLite), no sólo que el proceso esté vivo. El puerto publicado conecta el tráfico del host con el proceso `node` del contenedor.

Criterio SENA. CHECK-04 (genera la aplicación y el entorno de desarrollo).

14. FASE 09 — Modificación de la aplicación

Objetivo. Modificar el código de la aplicación para demostrar el ciclo de cambio y reconstrucción.

Procedimiento. Se localizó el mensaje de lista vacía en `src/static/js/app.js` y se editó únicamente ese texto.

Campo	Valor
Archivo	<code>app/getting-started-app/src/static/js/app.js</code>
Línea	56
Valor anterior	<code>No items yet! Add one above!</code>
Valor nuevo	<code>¡Aún no tienes elementos pendientes! ¡Agrega uno arriba!</code>

Evidencia. `evidencias/09-modificacion/modificacion-app.txt` (incluye `git diff`).

Figura 9. Modificación del código fuente en `app.js` línea 56.

Análisis técnico. El cambio se realizó sólo sobre la cadena requerida, sin alterar la lógica. Como el archivo forma parte de los estáticos servidos, el cambio debe propagarse mediante una **nueva imagen**; el contenedor en ejecución no lee el disco del host.

Criterio SENA. CHECK-05 (modifica con éxito la aplicación del contenedor).

15. FASE 10 — Reconstrucción de la imagen

Objetivo. Generar una nueva versión de la imagen con el cambio.

Comando.

```
docker build -f docker/Dockerfile -t getting-started app/  
getting-started-app
```

Resultado. Nuevo Image ID `3346675b07f4` , distinto del inicial `70d582dbf294` . Las capas de la imagen base y de dependencias se reutilizaron desde caché (`CACHED`); sólo la capa `COPY . .` se reconstruyó.

Evidencia. `evidencias/10-rebuild/docker-rebuild.txt` .

Figura 10. Reconstrucción de la imagen tras la modificación.

Análisis técnico. Docker etiqueta la nueva imagen con el mismo nombre (`getting-started:latest`), por lo que el tag apunta ahora a la versión modificada y la imagen antigua queda “colgando” (sin etiqueta). La caché de capas hace que la reconstrucción sea rápida.

Criterio SENA. CHECK-05 y CHECK-03.

16. FASE 11 — Reemplazo del contenedor anterior

Objetivo. Retirar el contenedor antiguo (que aún ejecuta la versión sin modificar).

Comandos.

```
docker stop 050918f28eba
docker rm 050918f28eba
docker ps
```

Resultado. El contenedor `050918f28eba` se detuvo y eliminó; `docker ps` ya no lo muestra y el puerto 3000 quedó libre.

Evidencia. `evidencias/11-reemplazo/docker-stop-rm.txt` .

Figura 11. Detención y eliminación del contenedor anterior.

Análisis técnico. No pueden coexistir dos contenedores publicando el **mismo puerto del host** (3000); el segundo `docker run -p 3000:3000` fallaría con `port is already allocated`. Por eso se detiene y elimina el anterior antes de crear el nuevo. `docker stop` envía SIGTERM (la app lo maneja con un graceful shutdown) y `docker rm` elimina el contenedor detenido. **No se ejecutaron comandos de limpieza masiva** (`docker system prune` , etc.) para no afectar otros proyectos del equipo.

Criterio SENA. CHECK-01 y CHECK-05.

17. FASE 12 — Nuevo contenedor y validación de la modificación

Objetivo. Ejecutar la versión actualizada y comprobar que el cambio es visible.

Comando.

```
docker run -dp 3000:3000 --name getting-started getting-started
```

Resultado. Nuevo contenedor `ba5968cb3faa` . La verificación del recurso servido confirmó el cambio:

```
$ curl -s http://localhost:3000/js/app.js | grep "elementos pendientes"
56: <p className="text-center">¡Aún no tienes elementos pendientes! ¡Agrega u

Cadena antigua "No items yet": ELIMINADA
GET / -> HTTP 200 | GET /items -> HTTP 200
```

Evidencia. evidencias/12-validacion/validacion-final.txt .

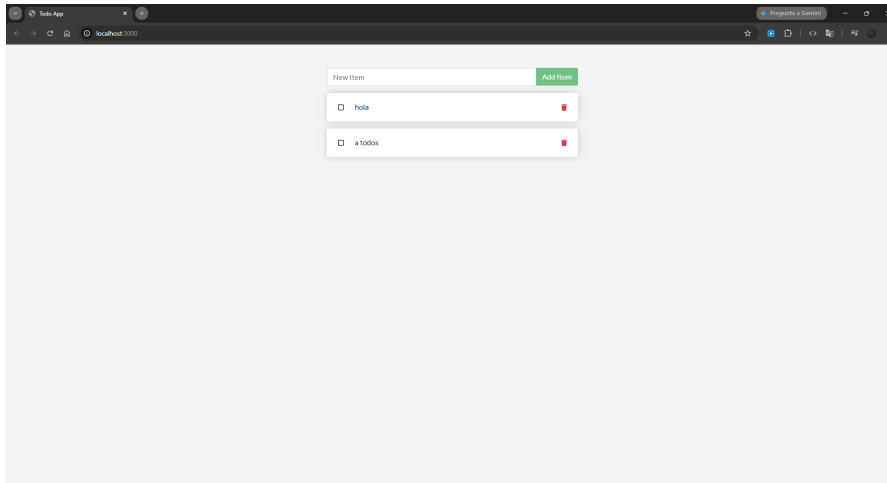


Figura 12. Interfaz Todo App servida por el contenedor final en `http://localhost:3000` , con tareas gestionadas desde el navegador.

Figura 12. Aplicación modificada ejecutándose dentro del nuevo contenedor (captura del navegador).

Análisis técnico. Esta es la prueba de extremo a extremo: el texto nuevo llega al navegador **desde el código fuente modificado, pasado por el Dockerfile, la nueva imagen y el nuevo contenedor**. Demuestra que la cadena de construcción propagó el cambio.

Criterio SENA. CHECK-05 (modificación exitosa).

18. FASE 13 — Docker Hub

Estado: NO REALIZADO (fase opcional).

La publicación en Docker Hub no es necesaria para la lista de chequeo y requiere autenticación interactiva. Conforme a la regla de seguridad, no se solicitan ni se almacenan credenciales. Si se desea, el aprendiz puede ejecutar manualmente:

```
docker login
docker tag getting-started <USUARIO>/getting-started
docker push <USUARIO>/getting-started
```

Se marca honestamente como **no realizado**; no constituye evidencia de cumplimiento.

19. FASE 14 — Pruebas finales (verificación automatizada)

Objetivo. Auditar automáticamente el despliegue completo.

Comando.

```
bash scripts/verify.sh
```

Resultado (real).

```
[PASS] Docker CLI disponible (Docker version 29.7.2)
[PASS] Docker Engine responde (docker info)
[PASS] Imagen 'getting-started' existe (ID sha256:3346675b07f4...)
[PASS] Contenedor 'getting-started' en ejecución (ID ba5968cb3faa)
[PASS] Puerto 3000 mapeado (host -> contenedor)
[PASS] Aplicación responde HTTP 200 en localhost:3000
[PASS] Modificación visible: '¡Aún no tienes elementos pendientes! ¡Agrega un
RESULTADO: 7 pruebas OK, 0 fallidas
ESTADO GLOBAL: VERIFICACIÓN EXITOSA
```

Evidencia. logs/final-verification.log .

Error encontrado y corregido: la primera versión del script usaba el patrón `grep "3000->"`, que no coincidía con la salida real de `docker port (3000/tcp -> 0.0.0.0:3000)`. Se corrigió el patrón a `3000/tcp ->` y la verificación pasó a 7/7. Este episodio se documenta como parte del aprendizaje (diagnóstico de un falso negativo, no de un fallo de despliegue).

20. Matriz de cumplimiento

Indicador	Peso	Evidencias	Resultado
CHECK-01 — Maneja instrucciones para la navegación en Docker	30%	EVID-03, EVID-04, EVID-07, EVID-09, EVID-13	CUMPLE
CHECK-02 — Maneja los conceptos de contenedores	30%	EVID-01, EVID-04, EVID-09 + explicaciones (imagen/ contenedor/ Dockerfile/ registry/ engine/ puerto/host)	CUMPLE
CHECK-03 — Genera con éxito la imagen	10%	EVID-05, EVID-06, EVID-07	CUMPLE
CHECK-04 — Genera la aplicación y	15%	EVID-02, EVID-08, EVID-10	CUMPLE

Indicador	Peso	Evidencias	Resultado
el entorno de desarrollo			
CHECK-05 — Modifica con éxito la aplicación del contenedor	15%	EVID-11, EVID-12, EVID-13, EVID-14, EVID-15	CUMPLE

Los identificadores corresponden a evidencias reales registradas en `metadata/evidence-index.json`.

21. Resultados obtenidos

1. Entorno Docker operativo verificado (Engine 29.7.2, Compose v5.3.1).
2. Imagen `getting-started` construida desde un Dockerfile propio (Node 22 Alpine).
3. Contenedor ejecutándose con la aplicación accesible en `http://localhost:3000`.
4. Funcionalidad CRUD comprobada (crear, listar, completar, eliminar).
5. Código modificado (`app.js:56`), imagen reconstruida (nuevo ID) y contenedor reemplazado.
6. Cambio visible en la aplicación final (prueba de extremo a extremo).
7. Verificación automatizada: 7/7 pruebas exitosas.
8. Paquete de evidencias y documentación trazable generado.

22. Conclusiones

- **Aprendizaje.** Se comprendió de forma práctica el ciclo de vida de una aplicación en contenedores: código fuente → Dockerfile → imagen → contenedor → puerto → aplicación.

- **Uso de Docker.** Se emplearon comandos de navegación y gestión (`docker build` , `image ls` , `run` , `ps` , `port` , `logs` , `stop` , `rm`) sobre una aplicación real.
- **Imagen vs. contenedor.** La imagen es una plantilla inmutable; el contenedor es su instancia en ejecución. Reconstruir una imagen **no** cambia el contenedor en marcha: fue necesario detener y eliminar el anterior y crear uno nuevo.
- **Importancia del Dockerfile.** Define de forma reproducible cómo se construye la imagen; separarlo del código (con `-f`) mantiene el proyecto ordenado.
- **Aislamiento.** El contenedor encapsula runtime y dependencias, evitando conflictos con el sistema (por ejemplo, la versión de Node del host, v24, no afectó a la imagen, Node 22).
- **Mapeo de puertos.** `-p 3000:3000` conecta el host con el contenedor; sin él, la aplicación sería inaccesible desde el navegador, y el mismo puerto no puede publicarse dos veces.
- **Modificación y reconstrucción.** El cambio en `app.js` sólo llegó a producción al reconstruir la imagen y reemplazar el contenedor, evidenciando la trazabilidad del despliegue.
- **Relación con infraestructura tecnológica.** Este proceso es la base de una plataforma de despliegue: imágenes versionadas, contenedores aislados, puertos controlados y verificación automatizada, alineados con el resultado de aprendizaje **220501086-02**.

23. Anexos

- **Anexo A. Registro de comandos:** `metadata/commands.log` .
- **Anexo B. Índice de evidencias:** `metadata/evidence-index.json` .
- **Anexo C. Estado del proyecto:** `metadata/project-state.json` .
- **Anexo D. Lista de chequeo:** `metadata/checklist.json` .
- **Anexo E. Script de verificación:** `scripts/verify.sh` y su salida en `logs/final-verification.log` .
- **Anexo F. Evidencias de terminal:** carpeta `evidencias/` .
- **Anexo G. Capturas de pantalla (reales, aportadas por el aprendiz):** `evidencias/02-docker/docker-version-info.png` ,

evidencias/02-docker/docker-info-server.png , evidencias/07-contenedor/docker-desktop-containers.png y evidencias/12-validacion/app-navegador.png . No se generan capturas sintéticas; todas las imágenes incluidas corresponden a capturas reales del entorno.